4

4.1

StarryOS

moss

`fork` COW

4.2

`fork`

`mprotect` moss `MemoryArea`

`Backend` `PageTable`

4-1

```
AddrSpace
├── va_range              可用虚  地址上下界
├── areas: MemorySet
│   └── BTreeMap<VirtAddr, MemoryArea>
│       ├── va_range
│       ├── flags           / 写 /      / 用      限
│       └── backend      Linear | Cow | Shared | File
└── pt: PageTable         已装入的虚     → 物理  快照
```

4-1 AddrSpace

4.2.1 AddrSpace MemoryArea

`ProcessData` `Arc<Mutex<AddrSpace>>`

`fork` `AddrSpace` `CLONE_VM`

clone

`AddrSpace`

```rust
pub struct AddrSpace {
    va_range: VirtAddrRange,   // 允  用    程使用的虚  地址范
    areas: MemorySet<Backend>, // 按起始地址    的虚  内存区域
    pt: PageTable,             // 已生效的虚    到物理  映射
}
```

4-2

MemorySet MemoryArea BTreeMap

MemoryArea Backend

```rust
pub struct MemoryArea<B: MappingBackend> {
    va_range: AddrRange<B::Addr>, // 一段 的虚 地址范
    flags: B::Flags,              //
    backend: B,                   // 决定 面从何 取得
}
```

4-3 MemoryArea

map unmap protect

populate_area handle_page_fault

4.2.2 Backend

Backend AddrSpace

COW Backend

```rust
#[enum_dispatch(BackendOps)]
pub enum Backend {
    Linear(LinearBackend), // 按固定偏移建立虚 地址到物理地址的映射
    Cow(CowBackend),       // 管理匿名 和私有映射，fork 后 行写 制
    Shared(SharedBackend), //
    File(FileBackend),     // 将文件 存 映射到用 地址空
}
```

4-4 Backend

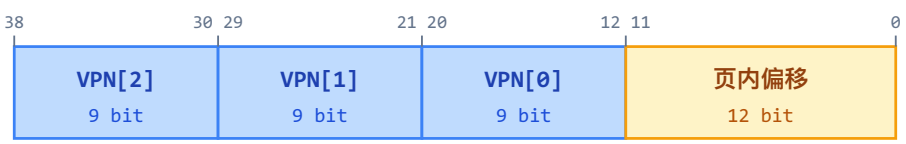
Backend populate clone_map

4.3

4.3.1

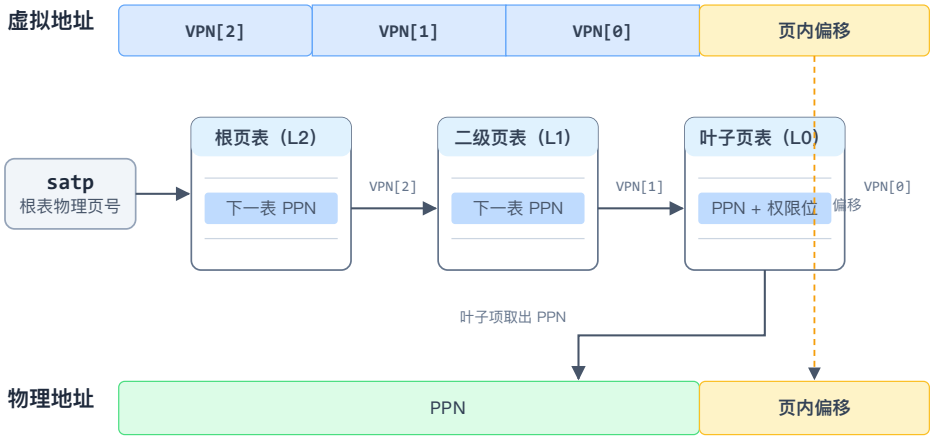
moss RISC-V64 LoongArch64

RISC-V64 Sv39 38



4-1 Sv39

satp VPN[2] VPN[1]
VPN[0] LoongArch64



4-2 Sv39 4 KiB

axhal PageTable PageTableMut

```rust
#[cfg(target_arch = "riscv64")]
pub type PageTable = Sv39PageTable<PagingHandlerImpl>;

#[cfg(target_arch = "loongarch64")]
pub type PageTable = LA64PageTable<PagingHandlerImpl>;

pub struct PageTable64<M, PTE, H> {
    root_paddr: PhysAddr,        // 根  表的物理地址
    _phantom: PhantomData<(M, PTE, H)>,
}
```

4-5

“ ”

Backend MemoryArea

MemoryArea

4.3.2

CPU TLB TLB

satp / CSR

TLB

CPU

——

Backend

4.4

MemoryArea

4.4.1

MappingFlags READ WRITE EXECUTE USER MemoryArea.flags

MemoryArea.flags Backend

SIGSEGV

USER

EXECUTE

RISC-V X

COW WRITE

COW

4.4.2 mprotect

mprotect ——

WRITE——— AddrSpace::protect

munmap

flags MemoryArea

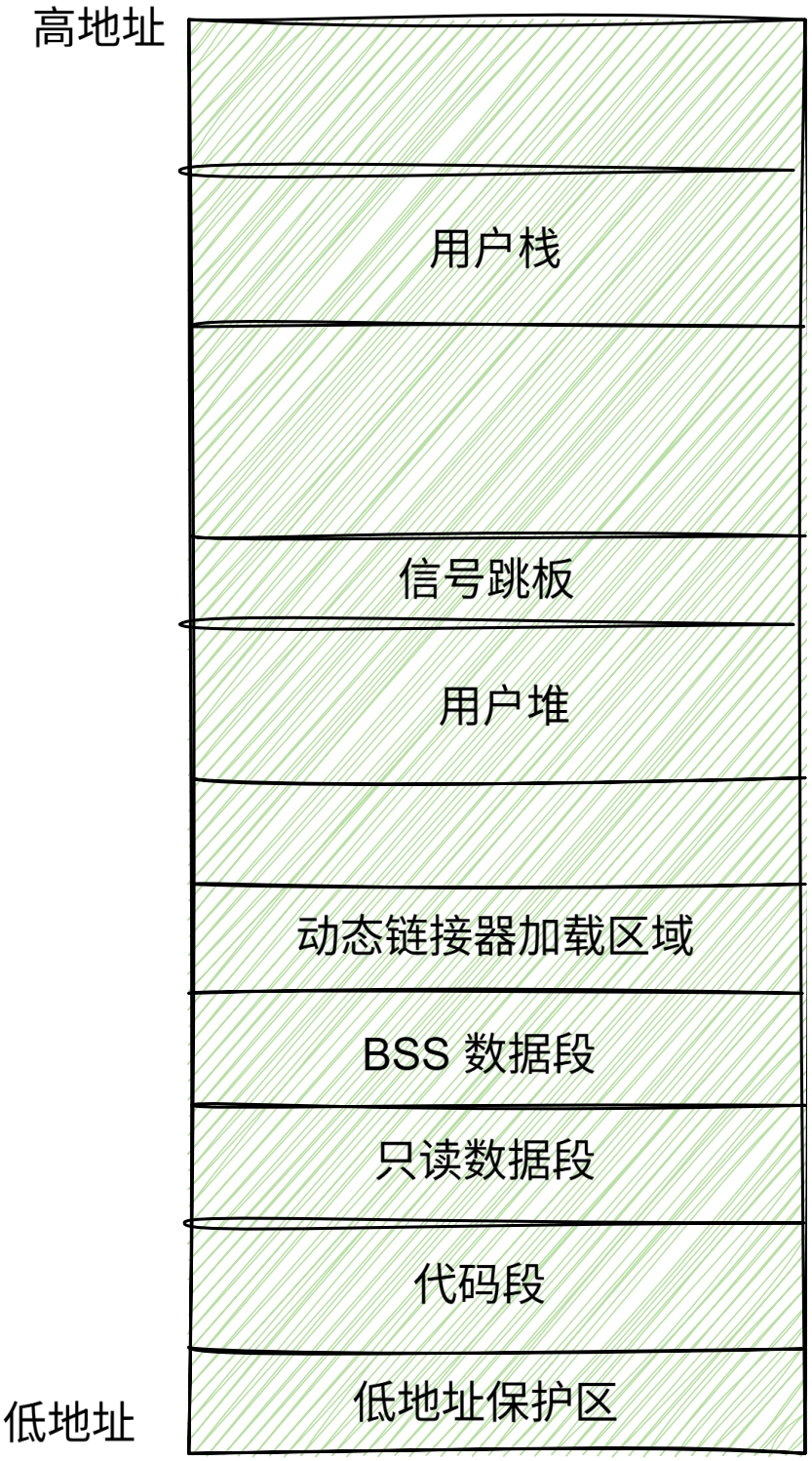
PROT_NONE

SIGSEGV MemoryArea

4.5

ELF

4-3 RISC-V64 LoongArch64 ELF



4-3 moss

4.5.1 ELF

ELF “ ” PT_LOAD

ELF

```
for ph in program_headers.iter().filter(|ph| ph.get_type() == Ok(Type::Load)) {
    let vaddr = ph.virtual_addr as usize + elf_base;
    let seg_start = VirtAddr::from_usize(vaddr);
```

```
    let seg_pad = vaddr.align_offset_4k();
    let seg_align_size = align_up_4k(ph.mem_size as usize + seg_pad);

    let backend = Backend::new_cow(
        seg_start,
        PageSize::Size4K,
        FileBackend::Cached(cache.clone()), // ELF 文件的  存  象
        ph.offset,
        Some(ph.offset + ph.file_size),     // 文件有效数据的  束位置
    );

    uspace.map(
        seg_start.align_down_4k(),
        seg_align_size,
        mapping_flags(ph.flags),
        false, // 只登  区域，不立即装入物理
        backend,
    )?;
}
```

4-6 ELF

Backend ELF

BSS

moss ELF `fork/exec`

BusyBox shebang

musl glibc

4.5.2

moss `USER_HEAP_BASE`

64 KiB `brk`

Cow Backend

`fork`

4.5.3

`mmap`

`MAP_FIXED`

`find_free_area`

	Backend	
MAP_PRIVATE | MAP_ANONYMOUS	Cow	
MAP_PRIVATE	Cow	
MAP_SHARED | MAP_ANONYMOUS	Shared	
MAP_SHARED	File	
	Linear	

4-1 mmap Backend

moss MAP_SHARED_VALIDATE MAP_FIXED_NOREPLACE MAP_GROWSDOWN

offset MAP_LOCKED mlock

rlimit moss mmap

/proc/[pid]/maps s p

mremap

MREMAP_MAYMOVE ——

MAP_PRIVATE | MAP_ANONYMOUS Backend

4.6

4.6.1

fork exec ——

exec

AddrSpace::map MemoryArea populate

populate_area ELF mmap MAP_POPULATE

populate_area MemoryArea Backend Backend

COW VirtMem

4.6.2

Cow Backend

Shared Backend `Option<PhysAddr>` `populate` `get_or_alloc` `SharedPages`

Linear Backend

4.6.3

File Backend “ + per-file LRU”

File Backend

```
// FileBackend 向   存注册回 ，在 存 被 逐   除用   表
let handle = cache.add_evict_listener({
    let this = Arc::downgrade(self);
    move |pn, _page| {
        let Some(this) = this.upgrade() else { return };
        this.on_evict(pn, &mut aspace);
    }
});
```

4-7 File Backend

Backend

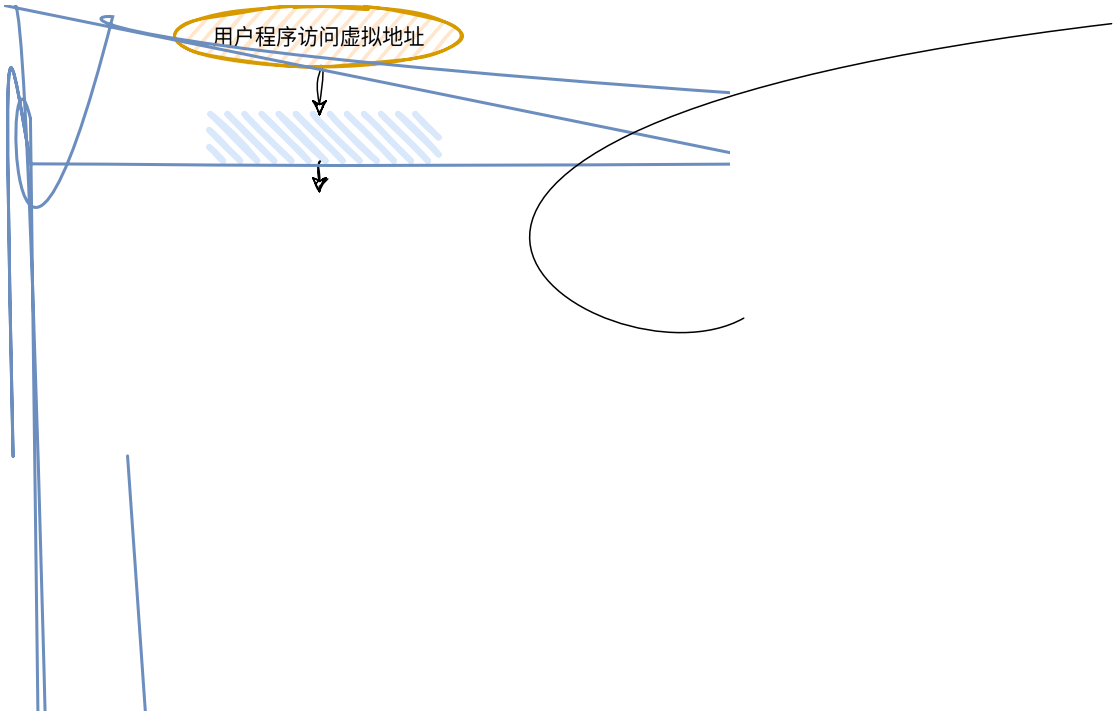
File Backend

4.6.4

4-4 CPU

`MemoryArea`

Backend


用户程序访问虚拟地址

grow-down		
	SIGSEGV	
	SIGBUS	

4-2

moss MAP_GROWSDOWN mremap SIGBUS LoongArch64 SIGBUS

4.7

4.7.1 fork

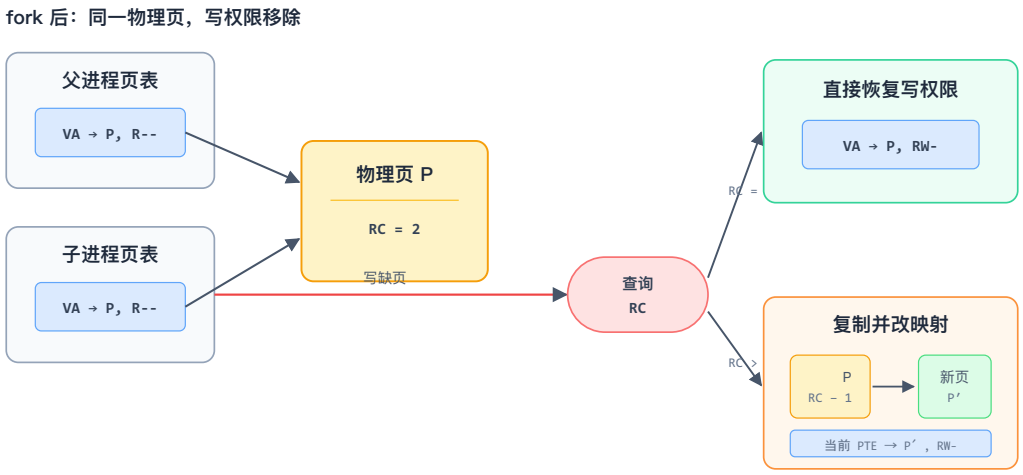
fork AddrSpace::try_clone

fork

CLONE_VM clone AddrSpace

4.7.2 COW

COW WRITE Cow Backend



4-5 COW

```
match frame_ref_count {
    1 => {
        // 已无其他地址空  共享，直接恢复写  限
        pt.protect(vaddr, flags)?;
    }
    _ => {
        // 仍被共享，复制数据并改映射到新  面
        let new_frame = alloc_new_frame(false)?;
        copy_page(paddr, new_frame);
        pt.remap(vaddr, new_frame, flags)?;
        drop_old_reference(paddr);
    }
}
```

4-9 COW

4.7.3 COW

StarryOS COW fork moss usize

moss Cow Backend

fork

fork/exec

4.8

munmap MemorySet MemoryArea

```
pub fn unmap(
    &mut self,
    start: B::Addr,
    size: usize,
    page_table: &mut B::PageTable,
) -> MappingResult {
    let range = AddrRange::try_from_start_size(start, size)
        .ok_or(MappingError::InvalidParam)?;

    //                        MemoryArea
    self.areas.retain(|_, area| {
```

```
        if area.va_range().contained_in(range) {
            area.unmap_area(page_table).unwrap();
            false
        } else {
            true
        }
    });

    // 左右  界的相交区域：收  或切分，各自同步清除  表
    ...
}
```

4-10 MemorySet

`mprotect` `mremap` `mprotect` `mremap` `unmap` `brk` `execve` `AddrSpace::clear`

`AddrSpace` `Drop` `clear` exec Backend

4.9

moss `AddrSpace` `MemoryArea` `PageTable` Backend Backend `USER` `EXECUTE` `mprotect` `fork` Backend COW COW

moss StarryOS `mmap` `MAP_LOCKED` `mremap` Backend `msync` `fsync` `sync` COW `usize` File Backend